Assignment 2 - Baseline Model, API, Docker and Airflow

Result

The main component from Assignment 1 is implemented as a reproducible text-classification service. It can train from a local CSV or the current MinIO object, expose predictions through FastAPI, run in a non-root Docker container, and refresh the training dataset through an Airflow DAG.

1. Baseline model

Implementation: `src/support_classifier/train.py`.

The scikit-learn pipeline consists of TF-IDF word unigrams/bigrams and class-balanced logistic regression. Training uses a stratified 75/25 split and a three-fold grid search over `C`. The fitted pipeline contains both preprocessing and classification, preventing training/serving skew.

The verified baseline run on the selected BANKING77 data produced:

Metric	Result
Rows	1,552
Intents	8
Accuracy	0.9716
Macro F1	0.9726
Selected `C`	2.0

The split is stratified with a fixed seed. These are offline academic-dataset results and should not be treated as proof of real-company generalization. The included 64-row seed file remains a network- independent smoke fixture; its expected macro F1 is about 0.55. The fitted BANKING77 model and detailed classification report are written to `artifacts/model.joblib` and `artifacts/model.metrics.json`.

Reproduce the network-independent smoke run:

```
PYTHONPATH=src python -m support_classifier.train \
  --data data/seed_support_queries.csv \
  --output artifacts/model.joblib \
  --no-mlflow
```

2. Prediction API

Implementation: `src/support_classifier/api.py`.

Endpoints:

Method/path	Purpose
`POST /predict`	Return intent, routing group, confidence, low-confidence flag and model source
`GET /health`	Readiness and loaded model information
`GET /metrics`	Request and low-confidence counters
`POST /reload`	Reload the current MLflow champion after promotion
`GET /docs`	Generated OpenAPI/Swagger demo

Request:

```
{"query": "I do not recognize this cash withdrawal"}
```

Response shape:

```
{
  "intent": "cash_withdrawal_not_recognised",
  "routing_group": "fraud",
  "confidence": 0.74,
  "low_confidence": false,
  "model_source": "models:/support-ticket-classifier@champion"
}
```

Input validation rejects strings shorter than three or longer than 2,000 characters. The service first resolves the MLflow `@champion` alias and falls back to the local artifact for an offline demo.

3. Docker container

`docker/Dockerfile` builds one reusable Python 3.12 application image for ingestion, training and serving. The runtime uses an unprivileged `appuser`, writes only to `/app/artifacts`, and starts Uvicorn on port 8000. `.dockerignore` keeps test files, reports, caches and local artifacts out of the build context.

The complete environment is defined in `docker-compose.yml`. For only the baseline path:

```
docker compose up --build minio minio-init ingest mlflow pipeline api gateway
curl -s http://localhost:8000/health
curl -s http://localhost:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"query":"My transfer is still pending"}'
```

4. Object storage and ingestion

Training data is stored at `s3://support-data/training/latest.csv` in MinIO. The ingestion code:

1. downloads the selected BANKING77 classes from a fixed Hugging Face revision;
2. falls back to the local seed dataset if the remote source is unavailable;
3. normalizes text and removes exact duplicates;
4. validates schema, nulls and class sizes;
5. uploads a normalized CSV through the S3-compatible API.

MinIO console: `http://localhost:9001` with course-demo credentials `minioadmin/minioadmin`. Production credentials must be injected as secrets rather than using these local defaults.

5. Airflow job

`dags/support_data_ingestion.py` defines the `support_data_ingestion` DAG:

- schedule: daily;
- catchup: disabled;
- retries: two with a two-minute delay;
- task: download, clean, validate and upload;
- storage: MinIO via the same environment variables as the other services.

Start the Airflow UI and scheduler:

```
docker compose up --build airflow-init airflow-webserver airflow-scheduler
```

Open `http://localhost:8080`, sign in with `admin/admin`, enable the DAG, and trigger it. The task log prints the source, S3 URI, row count and per-class counts.

6. Requirement traceability

Assignment requirement	Evidence
Create the main component	Training and prediction packages under `src/support_classifier`
Train a baseline	`train.py` and saved joblib artifact
Wrap it in an API	FastAPI `api.py`, OpenAPI `/docs`
Put it in Docker	`docker/Dockerfile` and Compose services
Demonstrate the API	Commands above and `reports/demo.md`
Store training data in DB/object storage	MinIO `support-data` bucket
Airflow job for new data	`support_data_ingestion` DAG

7. Limitations and next step

The academic-data score is not presented as production evidence. The next assignment logs experiments and artifacts to MLflow, registers each model version, and introduces a champion-challenger quality gate before serving.